

System execution context

Document #: P2079R2
Date: 2022-01-14
Project: Programming Language C++
Audience: SG1, LEWG
Reply-to: Lee Howes
<lwh@fb.com>
Ruslan Arutyunyan
<ruslan.arutyunyan@intel.com>
Michael Voss
<michaelj.voss@intel.com>

§ 1 Abstract

A `system_context` and `system_scheduler` that expose a simple parallel-forward-progress thread pool that may share and expose an underlying system thread pool and is intended to be the basic execution context and scheduler that we recommend to be used in combination with [P2300R4].

§ 2 Changes

§ 2.1 R2

- Significant redesign to fit in P2300 model.
- Strictly limit to parallel progress without control over the level of parallelism.
- Remove direct support for task groups, delegating that to `async_scope`.

§ 2.2 R1

- Minor modifications

§ 2.3 R0

- first revision

§ 3 Introduction

[P2300R4] describes a rounded set of primitives for asynchronous and parallel execution that give a firm grounding for the future. However, the paper lacks a standard execution context and scheduler. It has been broadly accepted that we need some sort of standard scheduler.

As noted in [P2079R1], an earlier revision of this paper, the earlier `static_thread_pool` had many shortcomings. This was removed from [P2300R4] based on that and other input.

This revision updates [P2079R1] to match the structure of [P2300R4]. It aims to provide a simple, flexible, standard execution context that should be used as the basis for examples. It is a minimal design, with few constraints, and as such should be efficient to implement on top of something like a static thread pool, but also on top of system thread pools where fixing the number of threads diverges from efficient implementation goals.

Lifetime management and other functionality is delegated to other papers, primarily to the `async_scope` defined in [P2519R0]. Unlike in earlier versions of this paper, we do not provide support for waiting on groups of tasks, delegating that to the separate `async_scope` design in [P2519R0], because that is not functionality specific to a system context.

The system context is of undefined size, supporting explicitly *parallel forward progress*. By requiring only parallel forward progress, any created parallel context is able to be a view onto the underlying shared global context. All instances of the `system_context` share the same underlying execution context. If the underlying context is a static thread pool, then all `system_contexts` should reference that same static thread pool. This is important to ensure that applications can rely on constructing `system_contexts` as necessary, without spawning an ever increasing number of threads. It also means that there is no isolation between `system_context` instances, which people should be aware of when they use this functionality. Note that if they rely strictly on parallel forward progress, this is not a problem, and is generally a safe way to develop applications.

The minimal extensions to basic parallel forward progress are to support fundamental functionality that is necessary to make parallel algorithms work:

- Cancellation: work submitted through the parallel context must be cancellable.
- Forward progress delegation: we must be able to implement a blocking operation that ensures forward progress of a complex parallel algorithm without special cases.

In addition, early feedback on the paper from Sean Parent suggested a need to allow the system context to carry no threads of its own, and take over the main thread. This led us to add the `execute_chunk` and `execute_all` capability that makes forward progress delegation explicit such that in addition to the system context being able to delegate work when it needs to, we can build code that directly requests delegation of work such that an event loop can be constructed around this.

An implementation of `system_context` *should* allow link-time replacement of the implementation such that the context may be replaced with an implementation that

compiles and runs in a single-threaded process or that can be replaced with an appropriately configured system thread pool by an end-user. We do not attempt to specify here the mechanism by which this should be implemented.

§ 4 Design

§ 4.1 system_context

The `system_context` creates a view on some underlying execution context supporting *parallel forward progress*. A `system_context` must outlive any work launched on it.

```
class system_context {
public:
    system_context();
    ~system_context();

    system_context(const system_context&) = delete;
    system_context(system_context&&) = delete;
    system_context& operator=(const system_context&) = delete;
    system_context& operator=(system_context&&) = delete;

    system_scheduler get_scheduler();
    implementation-defined_delegation_sender execute_chunk() noexcept;
    implementation-defined_delegation_sender execute_all() noexcept;
    size_t max_concurrency() noexcept;
};
```

- On construction, the `system_context` may initialize a shared system context, if it has not been previously initialized.
- To support sharing of an underlying system context, two `system_context` objects do not guarantee task isolation. If work submitted by one can consume the thread pool, that can block progress of another.
- The `system_context` is non-copyable and non-moveable.
- The `system_context` must outlive work launched on it. If there is outstanding work at the point of destruction, `std::terminate` will be called.
- The `system_context` must outlive schedulers obtained from it. If there are outstanding schedulers at destruction time, this is undefined behavior.
- `get_scheduler` returns a `system_scheduler` instance that holds a reference to the `system_context`.
- `execute_chunk` returns a sender that will:
 - If connected to a receiver that provides a scheduler in response to `get_delegatee_scheduler` report non-blocking if submitting work to that scheduler is non-blocking.
 - When `start()` an implementation-defined number of tasks will be moved from the `system_context`'s internal queue and scheduled on the delegatee

scheduler.

- Otherwise will report blocking.
 - When `start()` is called on its operation state it will execute a chunk of work of implementation-defined size provided by the `system_context`.
- If connected to a receiver that provides a `stop_token`, will respond to stop requests by not executing or delegating further tasks from its queue and completing with `set_done`.
- On success completes with the number of tasks that were executed or delegated.
- Completes immediately with the value 0 if no tasks are available to run.
- `execute_all` behaves like `execute_chunk` except that it will run or delegate all tasks in the context. After the sender returned by `execute_all` completes, and no tasks were added after that completion, a subsequent attempt must complete with 0
- `max_concurrency` will return a value representing the maximum number of threads the context may support. This is not a snapshot of the current number of threads, and may return `numeric_limits<size_t>::max`. If the return value is 0, then `execute_chunk` must be used by at least 1 thread to drive the context.

§ 4.2 `system_scheduler`

A `system_scheduler` is a copyable handle to a `system_context`. It is the means through which agents are launched on a `system_context`. The `system_scheduler` instance does not have to outlive work submitted to it.

```
class system_scheduler {
public:
    system_scheduler() = delete;
    ~system_scheduler();

    system_scheduler(const system_scheduler&);
    system_scheduler(system_scheduler&&);
    system_scheduler& operator=(const system_scheduler&);
    system_scheduler& operator=(system_scheduler&&);

    bool operator==(const system_scheduler&) const noexcept;

    friend implementation-defined-system_sender tag_invoke(
        std::execution::schedule_t, const system_scheduler&) noexcept;
    friend std::execution::forward_progress_guarantee tag_invoke(
        std::execution::get_forward_progress_guarantee_t,
        const system_scheduler&) noexcept;
    friend implementation-defined-bulk_sender tag_invoke(
        std::execution::bulk_t,
        const system_scheduler&,
        Sh&& sh,
        F&& f) noexcept;
};
```

- `system_scheduler` is not independently constructable, and must be obtained from a `system_context`. It is both move and copy constructable and assignable.
- Two `system_schedulers` compare equal if they share the same underlying `system_context`.
- A `system_scheduler` has reference semantics with respect to its `system_context`. Calling any operation other than the destructor on a `system_scheduler` after the `system_context` it was created from is destroyed is undefined behavior, and that operation may access freed memory.
- The `system_scheduler`:
 - satisfies the scheduler concept and implements the schedule customisation point to return an implementation-defined sender type.
 - implements the `get_forward_progress_guarantee` query to return `parallel`.
 - implements the bulk CPO to customise the bulk sender adapter such that:
 - When `execution::set_value(r, args...)` is called on the created receiver, an agent is created with parallel forward progress on the underlying `system_context` for each `i` of type `Shape` from `0` to `sh` that calls `f(i, args...)`.
- schedule calls on a `system_scheduler` are non-blocking operations.
- If the underlying `system_context` is unable to make progress on work created through `system_scheduler` instances, and the sender retrieved from scheduler is connected to a receiver that supports the `get_delegatee_scheduler` query, work may be scheduled on the scheduler returned by `get_delegatee_scheduler` at the time of the call to `start`, or at any later point before the work completes.

§ 4.3 Parallel sender

```
class implementation-defined-system_sender {
public:
    friend pair<std::execution::system_scheduler, delegatee_scheduler>
        tag_invoke(
            std::execution::get_completion_scheduler_t<set_value_t>,
            const system_scheduler&) noexcept;
    friend pair<std::execution::system_scheduler, delegatee_scheduler>
        tag_invoke(
            std::execution::get_completion_scheduler_t<set_done_t>,
            const system_scheduler&) noexcept;

    template<receiver R>
        requires receiver_of<R>
    friend implementation-defined-operation_state
        tag_invoke(execution::connect_t, implementation-defined-
            system_sender&&, R&&);

    ...
};
```

schedule on a `system_scheduler` returns some implementation-defined sender type.

This sender satisfies the following properties:

- Implements the `get_completion_scheduler` query for the value and done channel where it returns a type that is a pair of an object that compares equal to itself, and a representation of delegatee scheduler that may be obtained from receivers connected with the sender.
- If connected with a receiver that supports the `get_stop_token` query, if that `stop_token` is stopped, operations on which `start` has been called, but are not yet running (and are hence not yet guaranteed to make progress) **must** complete with `set_done` as soon as is practical.
- connecting the sender and calling `start()` on the resulting operation state are non-blocking operations.

§ 5 Examples

As a simple parallel scheduler we can use it locally, and `sync_wait` on the work to make sure that it is complete. With forward progress delegation this would also allow the scheduler to delegate work to the blocked thread. This example is derived from the Hello World example in [P2300R4]. Note that it only adds a well-defined context object, and queries that for the scheduler. Everything else is unchanged about the example.

```
using namespace std::execution;

system_context ctx;
scheduler auto sch = ctx.scheduler();

sender auto begin = schedule(sch);
sender auto hi = then(begin, []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = this_thread::sync_wait(add_42).value();
```

We can structure the same thing using `execution::on`, which better matches structured concurrency:

```
using namespace std::execution;

system_context ctx;
scheduler auto sch = ctx.scheduler();

sender auto hi = then(just(), []{
    std::cout << "Hello world! Have an int.";
    return 13;
});
sender auto add_42 = then(hi, [](int arg) { return arg + 42; });

auto [i] = this_thread::sync_wait(on(sch, add_42)).value();
```

The `system_scheduler` customises bulk, so we can use bulk dependent on the scheduler. Here we use it in structured form using the parameterless `get_scheduler` that retrieves the scheduler from the receiver, combined with on:

```
auto bar() {
    return
        ex::let_value(
            ex::get_scheduler(),           // Fetch scheduler from receiver.
            [](auto current_sched) {
                return bulk(
                    current_sched.schedule(),
                    1,                     // Only 1 bulk task as a lazy way of
                    making cout safe
                    [](auto idx){
                        std::cout << "Index: " << idx << "\n";
                    })
            });
}

void foo()
{
    using namespace std::execution;

    system_context ctx;
    scheduler auto sch = ;

    auto [i] = this_thread::sync_wait(
        on(
            ctx.scheduler(),               // Start bar on the system_scheduler
            bar()))                       // and propagate it through the
        receivers
        .value();
}
```

Use `async_scope` and the delegation functionality of the context to build a loop to drive the context. This will be important if the context has no threads and we have setup the system for a single-threaded process:

```
using namespace std::execution;

system_context ctx;

int result = 0;

{
    async_scope scope;
    scheduler auto sch = ctx.scheduler();

    sender auto work =
        then(just(), [&](auto sched) {

            int val = 13;

            auto print_sender = then(just(), [val]{
```

```

        std::cout << "Hello world! Have an int with value: " << val <<
        "\n";
    });

    // spawn the print sender on sched to make sure it
    // completes before shutdown
    scope.spawn(on(sch, std::move(print_sender)));

    return val;
});

scope.spawn(on(sch, std::move(work)));

    // Loop to drain the context and subsequently check that the scope is
    // empty
    // We need a repeat algorithm to do this correctly, the following logic
    // approximates what a repeat algorithm would achieve.
    while(this_thread::sync_wait(ctx.execute_chunk()).value != 0);
    this_thread::sync_wait(when_all(scope.empty(), ctx.execute_all()));
};

// The scope ensured that all work is safely joined, so result contains 13
std::cout << "Result: " << result << "\n";

// and destruction of the context is now safe

```

§ 6 References

[P2079R1] Ruslan Arutyunyan, Michael Voss. 2020-08-15. Parallel Executor.

<https://wg21.link/p2079r1>

[P2300R4] 2022. std::execution.

<https://wg21.link/p2300r4>

[P2519R0] 2022. async_scope - Creating scopes for non-sequential concurrency.

<https://wg21.link/p2519>